

```

#!/usr/bin/env python3
class Logikk:
    def evaluate(logisk_formel, sannhetsverdier):
        # forventer at logisk_formel er en funksjon som tar len(sannhetsverdier)
        # antall argumenter
        # f.eks: om logisk_formel = lambda p, q, r... osv: (p and q) or (r and
        # (not p))
        # kaller vi bare => logisk_formel(p, q, r...) som gir oss sant eller usant
        return logisk_formel(*sannhetsverdier) # *[p, q, r] = p, q, r : unpacker

    def sjekk_logisk_konsekvens(premisser, konklusjon, n):
        # alle kombinasjoner av 010101.. hvor tilordningen av disse
        sannhetsverdiene gjør alle premissene sanne
        gode_kombinasjoner = Logikk.kombinasjoner_premises_true(premisser, n)
        # returner om KONKLUSJONEN er SANN for ALLE kombinasjoner der PREMISSENE
        er sanne
        return all(Logikk.evaluate(konklusjon, sannhetsverdier) for
        sannhetsverdier in gode_kombinasjoner)

    def kombinasjoner_premises_true(premisser, n):
        # f.eks om vi har fire variabler, så er det 2^4 kombinasjoner.
        # Og alle kombinasjonene er gitt gjennom de binære-representasjonene av
        tallene i rekkevidden [0, 2^n) altså [0, 16).
        # siste kombinasjon er 1111 -> alle fire er sanne, og dette er ved tallet
        15.
        kombinasjoner = []
        for bitstreng in range(0, 1<<n): # bruker bare bitshift
            sannhetsverdier = Logikk.bit_to_list(bitstreng, n)
            if Logikk.all_premises_true(premisser, sannhetsverdier):
                kombinasjoner.append(sannhetsverdier)
        return kombinasjoner
    def all_premises_true(premisser, sannhetsverdier):
        return all(Logikk.evaluate(formel, sannhetsverdier) for formel in
        premisses)

    def bit_to_list(bitstreng, n): # gjør f.eks tallet 15, som i binært er 1111,
    om til listen [1, 1, 1, 1]
        return [True if bitstreng & (1<<i) else False for i in range(n-1, -1, -1)]

# tester litt:
premiss1 = lambda p1, p2: p1 # p1 er sann
premiss2 = lambda p1, p2: (not p1) or p2 # p1 impliserer p2
premisses = {premiss1, premiss2} # dette er det samme som {p1, p1 -> p2}
konklusjon = lambda p1, p2: p2 # Er konklusjon sann gitt premissene ovenfor?
# Det vi sjekker her er basicly: {premiss1, premiss2} |= konklusjon, som er det
samme som {p1, p1 -> p2} |= p2. Som skal være sant.
print(Logikk.sjekk_logisk_konsekvens(premisser, konklusjon, 2)) # 2 for antall
variabler som er med
# Funker som den skal!

```